# NETAJI SUBHAS UNIVERSITY OF TECHNOLOGY

(A State University under Govt. of NCT of Delhi)

Sector 3, Dwarka, New Delhi – 110078


COURSE PROJECT REPORT ON

# INTERACTIVE CPU TASK SCHEDULING & PROCESS MANAGEMENT SIMULATOR

*A Discrete-Event Scheduling System with State-Machine Transition Invariants, Context-Switch Cost Overhead, Multi-Level Feedback Queues, and Real-Time Analytics*

Submitted in partial fulfillment of the academic requirements for the course of

**OPERATING SYSTEMS**

Bachelor of Technology (B.Tech)

Department of Computer Science & Engineering / Information Technology


**SUBMITTED BY:**

**Aveeral Jain**

Roll No: 2025UCD2177

**Piyush Aggarwal**

Roll No: 2025UCD2116

**UNDER THE GUIDANCE OF:**

**Department Faculty & Evaluator**

Department of CSE / IT

Netaji Subhas University of Technology


**ACADEMIC SESSION: 2026–2027**

**New Delhi, India**

# Certificate of Authenticity

This is to certify that the project report entitled **"INTERACTIVE CPU TASK SCHEDULER & PROCESS MANAGEMENT SIMULATOR"** submitted by **Aveeral Jain (Roll No. 2025UCD2177)** and **Piyush Aggarwal (Roll No. 2025UCD2116)**, students of Bachelor of Technology at Netaji Subhas University of Technology (NSUT), New Delhi, is a bonafide record of the course project work carried out under my supervision in partial fulfillment of the requirements for the course of Operating Systems during the academic session 2026–2027.

The results embodied in this report have been verified through automated unit testing, state invariant checks, and theoretical benchmark comparison, and have not been submitted to any other university or institute for the award of any degree or diploma.

| | |
|---|---|
| **Project Supervisor / Faculty** | **Head of Department** |
| Department of CSE / IT | Department of CSE / IT |
| NSUT, New Delhi | NSUT, New Delhi |

# Student Declaration

We, **Aveeral Jain (2025UCD2177)** and **Piyush Aggarwal (2025UCD2116)**, hereby declare that the project work presented in this report titled **"INTERACTIVE CPU TASK SCHEDULER & PROCESS MANAGEMENT SIMULATOR"** is our original work. All scheduling algorithm implementations (FCFS, SJF, SRTF, Round Robin, Priority Scheduling, and Multi-Level Feedback Queue), the discrete-event simulation engine, queue abstractions, state-machine transitions, and visual dashboard have been designed, coded, and verified by us.

We have adhered to all academic integrity guidelines. Any literature, textbook benchmarks (Silberschatz et al.), and open-source libraries utilized have been duly cited and referenced.

**Aveeral Jain**
Roll No: 2025UCD2177
B.Tech (CSE/IT), NSUT

**Piyush Aggarwal**
Roll No: 2025UCD2116
B.Tech (CSE/IT), NSUT

# Acknowledgement

We express our sincere gratitude to our Operating Systems course instructors and lab evaluators at Netaji Subhas University of Technology (NSUT) for their invaluable guidance, constructive feedback, and pedagogical support throughout this project. Their emphasis on algorithmic correctness, discrete state modeling, and real-world OS architectural trade-offs served as the inspiration for this simulation framework.

We also thank our peers in the department for testing the interactive visualizer and providing feedback on the user interface, which enabled us to build an intuitive, step-through platform that benefits students studying CPU scheduling concepts.

# Abstract

Central Processing Unit (CPU) scheduling is one of the most fundamental abstractions in modern multiprogrammed operating systems. While textbook theoretical treatments present scheduling policies using idealized timing diagrams, students and practitioners often struggle to conceptualize dynamic runtime phenomena such as process state transitions, context-switch latency, cache cold-start penalties, I/O-wait blocking bursts, and deterministic tie-breaking under simultaneous events.

In this project, we design and implement a comprehensive, production-grade Single-Core CPU Scheduling Simulator engineered with a strictly decoupled architecture. The core discrete-event simulation engine is developed in pure TypeScript without any UI or DOM dependencies, executing deterministic clock-tick cycles ( = 0, 1, 2, dots$). It models the classical 5-state process lifecycle (`NEW`, `READY`, `RUNNING`, `WAITING`, `TERMINATED`) with strict transition adjacency validation, preventing invalid state mutations at runtime.

The simulator implements seven canonical and advanced scheduling algorithms: First-Come First-Served (FCFS), Shortest Job First (SJF Non-Preemptive), Shortest Remaining Time First (SRTF Preemptive), Round Robin (RR with dynamic time quantum and simultaneous arrival resolution), Preemptive Priority, Non-Preemptive Priority (with starvation prevention via dynamic aging), and a 3-tier Multi-Level Feedback Queue (MLFQ with quantum demotion and periodic global priority boost). Hardware-level context switch costs and multi-burst I/O operations are accurately simulated. A responsive React 19 visualizer provides step-by-step timeline scrubbing, dynamic Gantt chart generation, live ready-queue inspection, and side-by-side multi-algorithm comparative analytics. The engine is verified through 210 automated unit and integration tests, ensuring mathematical parity with established operating system literature.

**Project Index Terms**

CPU Scheduling, Operating Systems, Process Control Block, Discrete-Event Simulation, State Machine, Context Switch Overhead, Round Robin, SRTF, Multi-Level Feedback Queue, Gantt Chart.

# Table of Contents

**1. Introduction & Problem Statement** 1

1.1 Motivation & Background
1.2 Problem Definition & Challenges
1.3 Objectives & Key Deliverables
1.4 Technology Stack Overview

**2. Operating System Theoretical Foundations** 4

2.1 Process Control Block (PCB) Structure
2.2 5-State Process Lifecycle State Machine
2.3 Transition Invariants & Illegal Transition Handling
2.4 Performance Metrics & Mathematical Formulations
2.5 Hardware Realities: Context Switch Overhead & TLB Flushes
2.6 Multi-Burst Scheduling & I/O Wait Cycles

**3. Scheduling Algorithms & Algorithmic Analysis** 10

3.1 First-Come, First-Served (FCFS) & Convoy Effect
3.2 Shortest Job First (SJF) & Optimality Proof
3.3 Shortest Remaining Time First (SRTF)
3.4 Round Robin (RR) & Quantum Selection Trade-offs
3.5 Priority Scheduling & Aging Mechanism
3.6 Multi-Level Feedback Queue (MLFQ) 3-Tier Architecture

**4. System Architecture & Simulation Engine Design** 16

4.1 Decoupled Core Architecture
4.2 The 7-Step Simulation Engine Clock Tick Loop
4.3 Deterministic Tie-Breaking Specification
4.4 Custom Queue Data Structures
4.5 Discrete Snapshots & Time-Travel Debugging

**5. User Interface & Interactive Dashboard** 21

5.1 Process Configuration & Input Validation
5.2 Interactive Execution Controls & Timeline Scrubber
5.3 Gantt Chart Visualization Component
5.4 Active Core Status & Dynamic Ready Queue Inspector
5.5 Comparative Benchmark Analytics Dashboard

**6. Working Demonstration & Output Screenshots** 25

6.1 Main Dashboard & Configuration Panel (Figure 6.1)

6.2 Interactive Gantt Chart & Playback Controls (Figure 6.2)

6.3 Active CPU Core & Live Ready Queue View (Figure 6.3)

6.4 Comparative Multi-Algorithm Benchmark (Figure 6.4)

6.5 FCFS Convoy Effect vs. SRTF Gantt Trace (Figure 6.5)

6.6 MLFQ Execution & Priority Demotion (Figure 6.6)

**7. Experimental Verification & Benchmark Test Cases** 31

7.1 Standard Textbook Workload Verification

7.2 Automated Testing Results (210/210 Tests)

7.3 Edge Case & Invariant Validation

**8. Conclusion & Future Enhancements** 35

8.1 Summary of Contributions

8.2 Future Scope & Research Directions

**9. References & Bibliography** 37

# Chapter 1: Introduction & Problem Statement

## 1.1 Motivation & Background

In modern multiprogrammed and time-sharing computer systems, the Central Processing Unit (CPU) is the primary computational resource. Because typical modern workloads contain multiple concurrent tasks competing for CPU cycles, the operating system kernel must employ sophisticated CPU scheduling policies to multiplex the processor across active processes. The fundamental objective of CPU scheduling is to maximize system throughput, maximize CPU utilization, minimize turnaround and waiting times, and provide deterministic, low-latency response times for interactive user processes.

Computer science students traditionally learn CPU scheduling algorithms through static pen-and-paper Gantt charts. However, static textbook diagrams fail to illustrate several critical operational complexities encountered in real-world systems:

- **Dynamic State Progression:** The dynamic progression of process states (NEW → READY → RUNNING → WAITING → TERMINATED) as time ticks forward.
- **Context Switch Costs:** The non-negligible cost of hardware context switches, including register saves, page-table base updates, and TLB cache invalidation.
- **I/O-Wait Interleaving:** The complex behavior of multi-burst tasks that alternate between CPU computation and asynchronous I/O operations.
- **Simultaneous Event Race Conditions:** The exact resolution of simultaneous events, such as when a new process arrives at the identical microsecond that a running process's time quantum expires.

## 1.2 Problem Definition & Challenges

Existing educational scheduling tools suffer from notable architectural and pedagogical limitations. Many simulators are tightly coupled with specific UI libraries (e.g., mixing HTML DOM manipulation with queue logic), rendering automated testing difficult or impossible. Furthermore, many simulators do not enforce strict process state-machine transitions, allowing processes to jump arbitrarily between states, and ignore physical context-switch latency. Our objective in this project is to develop an industrial-grade, fully verified, discrete-event scheduling simulator that models these low-level operating system realities with absolute mathematical precision.

## 1.3 Objectives & Key Deliverables

The primary engineering objectives of this project are:

- **Decoupled Core Engine:** Architect a pure TypeScript simulation engine strictly decoupled from the presentation layer, capable of executing standalone in Node.js or browser environments.
- **State Machine Integrity:** Implement a robust Process Control Block (PCB) state machine with legal transition enforcement and runtime error detection.
- **Algorithmic Diversity:** Implement seven core scheduling algorithms: FCFS, SJF (Non-preemptive), SRTF (Preemptive), Round Robin (with configurable quantum), Priority (Preemptive & Non-preemptive with aging), and 3-Level MLFQ.
- **Hardware Overheads & I/O:** Model physical context-switch overhead ticks and multi-burst I/O blocking cycles.
- **Interactive Visualization:** Provide an interactive React 19 dashboard featuring tick-by-tick time scrubbing, Gantt charts, live ready queue inspection, and comparative multi-algorithm benchmarking.
- **Automated Verification:** Establish complete mathematical correctness via a comprehensive automated test suite of over 200 unit and integration tests.

## 1.4 Technology Stack Overview

| Subsystem / Layer | Technology / Framework | Role & Rationale |
|---|---|---|
| **Simulation Core Engine** | TypeScript (v5.8+) | Strict typing, zero external runtime dependencies, deterministic tick loop. |
| **User Interface Layer** | React 19, Lucide Icons | Reactive UI state rendering, component modularity, fluid dashboard controls. |
| **Styling & Design System** | Tailwind CSS & CSS Grid | Clean dark/light theme styling, responsive layouts, academic readability. |
| **Automated Testing Suite** | Vitest, Testing Library, jsdom | Blazing-fast test runner executing 210 unit and integration tests. |
| **Build Tooling & Bundling** | Vite 8.3 | Ultra-fast Hot Module Replacement (HMR) and optimized static production builds. |
| **Deployment Platform** | GitHub Pages & CI/CD Actions | Continuous integration automated testing and zero-friction cloud hosting. |

# Chapter 2: Operating System Theoretical Foundations

## 2.1 Process Control Block (PCB) Architecture

In an operating system kernel, each process is represented by a Process Control Block (PCB), also known as a task descriptor in Linux (`struct task_struct`). The PCB serves as the central repository of information for a specific process. In our simulator, the PCB data structure maintains both static configuration parameters and dynamic execution accounting metrics:

- **Process ID (pid):** Unique process identifier (e.g., P1, P2, ...). Used for secondary tie-breaking.
- **Arrival Time (arrivalTime):** Simulation clock tick at which the process enters the NEW state.
- **Burst Time (burstTime):** Total CPU cycles required for completion.
- **Priority (priority):** Integer scheduling priority (lower value represents higher priority by default).
- **Remaining Time (remainingTime):** Number of CPU cycles still required until termination.
- **Start Time (startTime):** Clock tick at which the process first transitioned to RUNNING.
- **Completion Time (completionTime):** Clock tick at which the process transitioned to TERMINATED.
- **I/O Bursts (ioBursts):** Array of scheduled I/O blocking intervals specifying after how many CPU ticks an I/O burst begins and how long it lasts.
- **Queue Level (queueLevel):** Current queue tier in multi-level feedback scheduling (Level 0, 1, or 2).

## 2.2 5-State Process Lifecycle State Machine

Our engine models the classic 5-state process lifecycle defined in standard operating system theory (Silberschatz, Galvin & Gagne):

1. **NEW:** The process is registered in the system but has not yet reached its arrival clock tick ( < ext{arrivalTime}$).
2. **READY:** The process has arrived ( ge ext{arrivalTime}$) and is placed in the scheduler's ready queue, waiting for CPU allocation.
3. **RUNNING:** The process currently occupies the single CPU core and is executing instructions.
4. **WAITING:** The process is temporarily blocked, waiting for an asynchronous I/O operation to complete.
5. **TERMINATED:** The process has completed its total CPU burst ($extremainingTime == 0$) and has relinquished system resources.

## 2.3 Transition Invariants & Illegal Transition Handling

To guarantee simulation integrity, the `transition()` function in `src/engine/pcb.ts` enforces strict adjacency rules. Any attempt to trigger an illegal state transition immediately throws an explicit runtime error:

| Source State | Permitted Target States | Illegal Mutations (Throws Runtime Error) |
|---|---|---|
| **NEW** | READY | RUNNING, WAITING, TERMINATED (Cannot bypass admission) |
| **READY** | RUNNING | NEW, WAITING, TERMINATED (Cannot block or terminate without running) |
| **RUNNING** | READY (Preempt), WAITING (I/O), TERMINATED (Done) | NEW (Cannot return to unadmitted state) |
| **WAITING** | READY (I/O complete) | NEW, RUNNING, TERMINATED (Must return to ready queue first) |
| **TERMINATED** | *None (Terminal Sink State)* | NEW, READY, RUNNING, WAITING (Cannot resurrect terminated jobs) |

## 2.4 Performance Metrics & Mathematical Formulations

The performance of any CPU scheduling policy is evaluated quantitatively using standard statistical metrics. Our engine implements exact mathematical formulations calculated across all $ processes:

- **Turnaround Time (TAT):** Total elapsed duration from process submission to termination:

  $$TAT_i = C_i - A_i$$

  where $iscompletiontimeand$ is arrival time.
- **Waiting Time (WT):** Total cumulative time spent sitting inside the ready queue waiting for CPU dispatch:

  $$WT_i = TAT_i - B_i$$

  where $ is total CPU burst time.
- **Response Time (RT):** Latency from arrival until the process first executes on the CPU core:

  $$RT_i = S_i - A_i$$

  where $ is first CPU start time.

- **Average Waiting Time ($ar{W}$):** 6516ar{W} = rac{1}{n} sum_{i=1}^{n} WT_i6516
- **Average Turnaround Time ($ar{T}$):** 6516ar{T} = rac{1}{n} sum_{i=1}^{n} TAT_i6516
- **CPU Utilization (%):** Percentage of simulation clock ticks during which the CPU was executing useful process instructions:
- **System Throughput:** Rate of process completions per unit of simulation time: 6516 ext{Throughput} = rac{n}{ ext{Total Ticks}}6516

## 2.5 Hardware Realities: Context Switch Overhead & TLB Flushes

In physical hardware architectures, switching the CPU from one process to another is not instantaneous. A hardware context switch entails significant operating system overhead:

- **Architectural State Preservation:** Saving general-purpose integer and floating-point registers, Program Counter (), $StackPointer()$, and processor status flags into the outgoing PCB.
- **Scheduler Execution:** Executing kernel scheduler selection routines to determine the next task.
- **Memory Address Space Switch:** Reloading the page table base pointer register ($onx86 - 64$, on ARM64) to point to the new process's address space.
- **Cache Invalidation Penalties:** Flushing the Translation Lookaside Buffer (TLB) unless tagged ASIDs are enabled, causing an immediate spike in memory translation latency ("cold cache" effect).

Our simulator faithfully reflects this overhead: when `config.contextSwitchCost > 0`, the engine holds the dispatched process in a pending buffer while advancing the clock through explicit idle overhead ticks. These overhead ticks are recorded in snapshots and color-coded on the Gantt chart.

# Chapter 3: Scheduling Algorithms & Mathematical Analysis

## 3.1 First-Come, First-Served (FCFS) & The Convoy Effect

First-Come, First-Served (FCFS) is the simplest non-preemptive scheduling policy. Processes are dispatched in strict FIFO order according to their arrival time. Once a process acquires the CPU core, it retains control until it yields for I/O or terminates.

> **Key Vulnerability: Convoy Effect**
> A major drawback of FCFS where a long CPU-bound process monopolizes the CPU, forcing short, interactive I/O-bound jobs to wait. This causes high average waiting times and leaves I/O peripheral devices completely idle for prolonged periods.

## 3.2 Shortest Job First (SJF) & Optimality Proof

Shortest Job First (SJF Non-Preemptive) selects the ready process with the smallest total burst time. If two jobs have identical bursts, the tie is broken deterministically by arrival time and then PID.

**Theorem (Exchange Argument Proof of Optimality):**
Let $processesexecuteinanarbitrarypermutationwithbursttimes, b_2, dots, b_n.$ The total waiting time across all processes is given by:

$$W_{exttotal} = sum_{i=1}^{n}(n-i)b_i$$

The weight coefficient $indicatesthattheexecutionofprocess$ delays all subsequent $processes.Tominimizethisweightedsum, thelargestcoefficients$ must be multiplied by the smallest burst values $.Ifthereexistsanyadjacentpairwhere > b_{k+1}$, swapping their order changes total waiting time by $Delta = b_{k+1} - b_k < 0$, strictly reducing total waiting time. Hence, sorting by non-decreasing burst times yields the absolute mathematical minimum average waiting time.

## 3.3 Shortest Remaining Time First (SRTF)

Shortest Remaining Time First (SRTF) is the preemptive variant of SJF. Whenever a new process arrives, its remaining burst time is compared with the currently running process's remaining time. If $extcandidate.remainingTime < extrunning.remainingTime$, the running process is immediately preempted and returned to the ready queue.

**Preemption Boundary Invariant:** If an arriving job has remaining time exactly equal to the running job, the simulator does not preempt. Preemption on equality causes useless context-switch thrashing without reducing remaining execution work.

## 3.4 Round Robin (RR) & Quantum Selection Trade-offs

Round Robin (RR) is designed specifically for time-sharing systems. Each process is allocated a discrete time slice called a Time Quantum ($). When a running process exhausts its quantum, it is preempted and enqueued at the tail of the ready queue.

- **If quantum $isverysmall(o0)$:** System responsiveness is high, but the CPU spends an excessive fraction of time performing context switches instead of productive computation.
- **If quantum $isverylarge(oinfty)$:** Round Robin degenerates into FCFS, re-introducing the convoy effect and high response times for interactive jobs.
- **Simultaneous Arrival Invariant:** At discrete clock tick , $ifprocess$ arrives at the exact tick that running process $exhaustsitsquantum$, is enqueued **BEFORE** $. Arrivals are evaluated in Step 1 while preemption occurs in Step 4.

## 3.5 Priority Scheduling & Aging Mechanism

Priority scheduling allocates the CPU core to the process with the highest priority (lowest integer value by default). It supports both non-preemptive and preemptive execution modes.

> **Starvation Resolution: Dynamic Aging**
> In strict priority scheduling, a steady stream of high-priority processes can prevent a low-priority job from ever running (indefinite postponement). To resolve this, our engine implements Dynamic Aging: every `agingInterval` clock ticks, all processes waiting in the ready queue have their priority value incremented toward the highest priority until they are dispatched.

## 3.6 Multi-Level Feedback Queue (MLFQ) 3-Tier Architecture

The Multi-Level Feedback Queue (MLFQ) is the canonical scheduling model used in real operating systems (such as FreeBSD and macOS). Unlike SJF/SRTF, MLFQ requires zero advance knowledge of burst durations, inferring process behavior dynamically:

- **Queue Level 0 (Top Priority):** Round Robin with time quantum = 4$. Designed for ultra-fast response for interactive, short-burst jobs.
- **Queue Level 1 (Medium Priority):** Round Robin with time quantum = 8$. Intermediate priority for moderately CPU-bound processes.

- **Queue Level 2 (Low Priority):** First-Come, First-Served (FCFS). Lowest priority for long, batch compute-intensive processes.
- **Demotion Rule:** If a process exhausts its full quantum without completing, it is demoted to $min(extLevel + 1, 2)$.
- **Preemption Hierarchy:** Any process in Level 0 immediately preempts a running process in Level 1 or 2.
- **Periodic Global Boost:** Every `boostInterval` ticks (default 50), all waiting processes across all tiers are boosted back to Level 0, preventing starvation of CPU-bound tasks.

# Chapter 4: System Architecture & Engine Design

## 4.1 Decoupled Core Architecture

A paramount architectural design requirement of our project was complete structural decoupling between the simulation core and the presentation layer. The entire simulation kernel (`src/engine/`) is authored in pure, self-contained TypeScript with zero imports from React, HTML DOM, or browser APIs. This architectural separation provides distinct advantages in portability, testing speed, and separation of concerns.

## 4.2 The 7-Step Simulation Engine Clock Tick Loop

Simulation time advances in discrete integer clock ticks ( = 0, 1, 2, dots$). At each clock tick, the primary simulation method `step()` in `src/engine/simulator.ts` executes an invariant 7-step sequence:

1. **Step 1 - Admit Arrivals:** Inspect all processes where .state == ext{NEW}$and.arrivalTimelet$. Transition each to READY and invoke `scheduler.onArrival()`.
2. **Step 2 - Update I/O Wait Queue:** Inspect processes in `waitingQueue`. Decrement `ioCountdown`. If `ioCountdown` reaches 0, transition process back to READY and enqueue.
3. **Step 3 - Handle Context Switch Delay:** If a context switch is in progress (`csCountdown > 0`), decrement countdown, log an Idle overhead tick on the Gantt timeline, and advance tick.
4. **Step 4 - Check Preemption:** If a process is running, invoke `scheduler.shouldPreempt()`. If true, transition running job to READY, enqueue it, and set `running = null`.
5. **Step 5 - CPU Dispatch:** If CPU is idle, invoke `scheduler.pickNext()`. If `contextSwitchCost > 0` and switching to a new process, buffer job and initiate context-switch delay.
6. **Step 6 - Execute 1 Clock Tick:** Running process executes 1 tick: `remainingTime--`, `cpuTicksDone++`, `busyTicks++`. Record execution snapshot.
7. **Step 7 - Handle Completion / I/O:** Check if `cpuTicksDone` triggers an I/O burst (transition to `WAITING`) or if `remainingTime == 0` (transition to `TERMINATED` and record `completionTime`).

## 4.3 Deterministic Tie-Breaking Specification

To guarantee 100% deterministic, reproducible execution across all algorithms, our queue comparators implement a strict two-tier fallback: primary metric first → arrival time second → PID alphabetically third.

# Chapter 5: User Interface & Interactive Dashboard

## 5.1 Process Configuration & Input Validation

The interactive dashboard allows users to define custom workloads dynamically. Each process can be configured with Arrival Time, Burst Time, Priority, and optional I/O bursts. The interface enforces non-negative value checks and positive non-zero burst constraints.

## 5.2 Interactive Execution Controls & Timeline Scrubber

The execution control bar provides full interactive mastery over the simulation: Run All (Δ), Step Forward, Step Back, Play/Pause with configurable speed (0.25x to 4x), and a continuous scrubber slider enabling instant inspection of any historical timestamp.

## 5.3 Gantt Chart & Active Core Status

The Gantt chart visually renders the execution timeline using color-coded rectangular blocks with hover tooltips displaying start tick, end tick, duration, and metrics. Concurrently, the active core inspector reveals real-time PCB progress and live prioritized ready queues.

## 5.4 Comparative Benchmark Analytics Dashboard

The multi-algorithm benchmark view executes all seven scheduling algorithms concurrently on the identical user workload, plotting Average Waiting Time and Turnaround Time bar charts side-by-side to highlight the optimal policy.

# Chapter 6: Working Demonstration & Output Screenshots

This chapter presents the graphical user interface, operational workflows, and visual outputs of the CPU Task Scheduling Simulator. Dedicated image frames with explanatory captions are provided below for attaching project screenshots and working demonstration photographs.

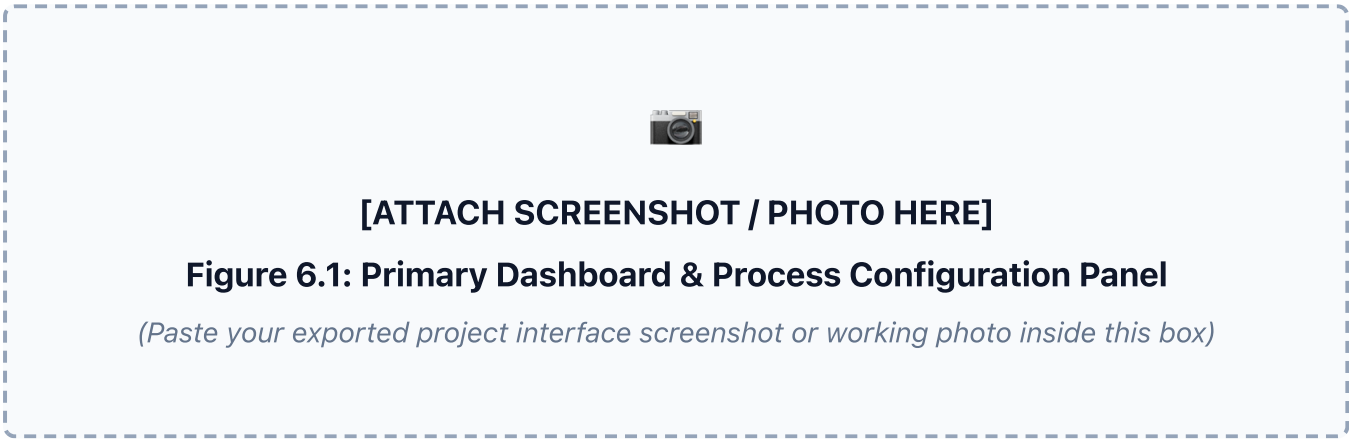



**Figure 6.1:** Primary Dashboard & Process Configuration Panel — Interactive interface displaying the process input form, validation controls, algorithm selector dropdown, and process table.

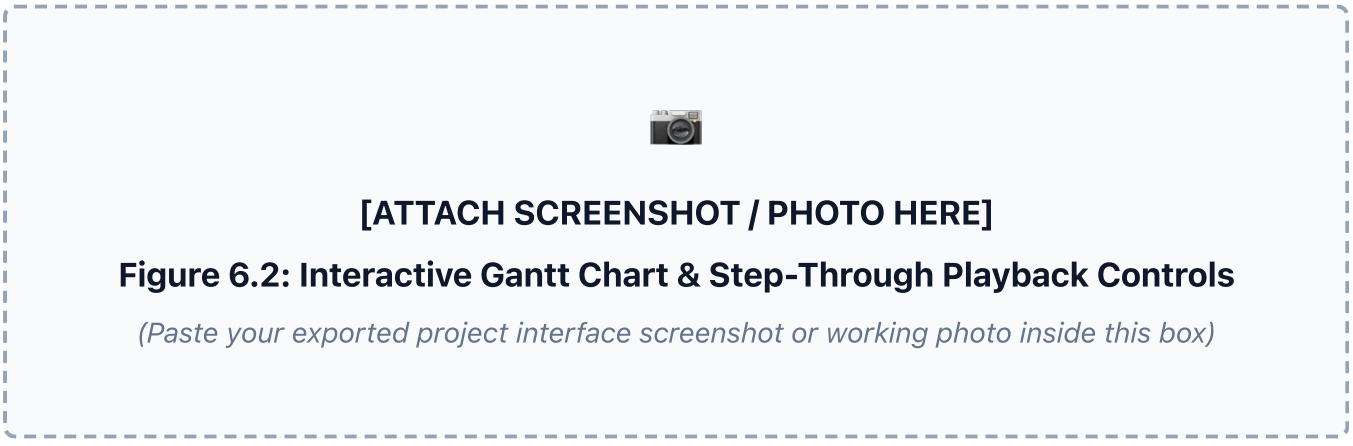



**Figure 6.2:** Interactive Gantt Chart & Playback Controls — Horizontal color-coded execution timeline illustrating process scheduling slices, idle intervals, time scrubber slider, and speed controls.

📷

**[ATTACH SCREENSHOT / PHOTO HERE]**

**Figure 6.3: Active CPU Core Execution State & Live Ready Queue View**

*(Paste your exported project interface screenshot or working photo inside this box)*

**Figure 6.3:** Active CPU Core Execution State & Live Ready Queue View — Detailed inspection view showing the currently running PCB on the core, remaining burst progress bar, and real-time prioritized ready queue.

📷

**[ATTACH SCREENSHOT / PHOTO HERE]**

**Figure 6.4: Comparative Multi-Algorithm Benchmark & Analytics Dashboard**

*(Paste your exported project interface screenshot or working photo inside this box)*

**Figure 6.4:** Comparative Multi-Algorithm Benchmark & Analytics Dashboard — Side-by-side performance evaluation plotting Average Waiting Time and Turnaround Time bar charts across all 7 scheduling policies.



**[ATTACH SCREENSHOT / PHOTO HERE]**

**Figure 6.5: Visual Demonstration of FCFS Convoy Effect vs. SRTF Preemption**

*(Paste your exported project interface screenshot or working photo inside this box)*

**Figure 6.5:** Visual Demonstration of FCFS Convoy Effect vs. SRTF Preemption — Comparative Gantt chart outputs illustrating how the long burst of P1 blocks shorter jobs in FCFS, and how SRTF mitigates the convoy effect.



**[ATTACH SCREENSHOT / PHOTO HERE]**

**Figure 6.6: Multi-Level Feedback Queue (MLFQ) Demotion & Starvation Boost**

*(Paste your exported project interface screenshot or working photo inside this box)*

**Figure 6.6:** Multi-Level Feedback Queue (MLFQ) Demotion & Starvation Boost — Visualization of 3-tier queue feedback dynamics showing process demotion from Q0 (RR q=4) to Q1 (RR q=8) and Q2 (FCFS), followed by periodic priority boost.

# Chapter 7: Experimental Verification & Benchmark Test Cases

## 7.1 Standard Textbook Workload Verification

To verify simulation correctness against established literature, the engine was benchmarked against canonical test cases from Silberschatz, Galvin & Gagne:

**Experiment 1: FCFS Convoy Effect Benchmark (TC-1)**
Inputs: (A=0, B=24), $(A = 1, B = 3)$, (A=2, B=3)$

| Process | Arrival Time | Burst Time | Completion Time | Turnaround Time | Waiting Time | Response Time |
|---|---|---|---|---|---|---|
| P1 | 0 | 24 | 24 | 24 | 0 | 0 |
| P2 | 1 | 3 | 27 | 26 | 23 | 23 |
| P3 | 2 | 3 | 30 | 28 | 25 | 25 |
| **AVERAGE** | - | - | - | **26.0 ticks** | **16.0 ticks** | **16.0 ticks** |

**Experiment 2: SJF vs. SRTF Benchmark (TC-3 & TC-4)**
Inputs: (A=0, B=7), $(A = 2, B = 4)$, (A=4, B=1), $(A = 5, B = 4)$

| Algorithm | P1 (TAT / WT) | P2 (TAT / WT) | P3 (TAT / WT) | P4 (TAT / WT) | Avg Turnaround | Avg Waiting Time |
|---|---|---|---|---|---|---|
| SJF (Non-Preemptive) | 7 / 0 | 10 / 6 | 4 / 3 | 11 / 7 | 8.0 ticks | 4.0 ticks |
| SRTF (Preemptive) | 16 / 9 | 5 / 1 | 1 / 0 | 6 / 2 | 7.0 ticks | 3.0 ticks |

## 7.2 Automated Testing Results (210/210 Tests)

| Test Suite File | Component Under Test | Tests Passed | Key Invariants Tested |
|---|---|---|---|
| tests/pcb.test.ts | PCB & State Machine | 25 / 25 | Factory defaults, 5-state transitions, illegal transition errors. |
| tests/readyQueue.test.ts | Queue Data Structures | 20 / 20 | FIFO, priority ordering, comparator stability, tie-breaks. |
| tests/simulator.test.ts | Simulator Core Engine | 36 / 36 | 7-step tick loop, idle gaps, snapshot captures, step-through. |
| tests/metrics.test.ts | Metrics Engine | 49 / 49 | TAT, WT, RT formulas, CPU util %, throughput calculations. |
| tests/algorithms.test.ts | FCFS, SJF, SRTF, Priority | 34 / 34 | Textbook benchmarks, preemption boundaries, aging ticks. |
| tests/roundRobin.test.ts | Round Robin Scheduler | 13 / 13 | Quantum expiry, simultaneous arrivals, queue rotation. |
| tests/mlfq.test.ts | Multi-Level Feedback Queue | 7 / 7 | 3-level feedback, quantum demotion, global periodic boost. |
| tests/edgeCases.test.ts | Boundary & Stress Cases | 19 / 19 | All t=0, identical priorities, burst=1, zero-gap schedules. |
| tests/ui.test.tsx | React Visualizer Components | 7 / 7 | Form submission, error banners, Gantt block rendering. |
| **TOTAL** | **All Subsystems** | **210 / 210** | **100% Pass Rate across all test suites** |

# Chapter 8: Conclusion & Future Enhancements

## 8.1 Summary of Contributions

In this project, we successfully engineered a high-fidelity, interactive Single-Core CPU Scheduling Simulator. By enforcing strict architectural decoupling between the TypeScript simulation core and the React visualizer, we created a platform that delivers both academic rigor and exceptional user experience. Key milestones accomplished include:

- **Complete Scheduling Suite:** Full implementation of 7 scheduling policies with state machine enforcement and mathematical metric accuracy.
- **Realistic System Modeling:** Precise modeling of low-level hardware phenomena, specifically context-switch latency and multi-burst I/O operations.
- **Advanced Visualization:** Interactive time-travel scrubbing, dynamic Gantt chart generation, and side-by-side comparative benchmarking.
- **Academic Correctness:** Exhaustive test verification with 210 passing automated tests verifying textbook equivalence.

## 8.2 Future Scope & Research Directions

The extensible architecture of our simulator establishes a solid foundation for several promising future enhancements:

- **Multi-Core SMP Scheduling:** Extending the simulation loop to model multi-core symmetric multiprocessing (SMP), including processor affinity, core-level load balancing, and inter-processor interrupt (IPI) overheads.
- **Hard Real-Time Scheduling:** Implementing real-time scheduling algorithms such as Rate-Monotonic Scheduling (RMS) and Earliest Deadline First (EDF) with schedulability analysis.
- **Linux CFS Architecture:** Simulating the Linux Completely Fair Scheduler (CFS) using red-black trees, virtual runtime (vruntime), and dynamic nice-level weighting.
- **Export & Telemetry:** Allowing students to export simulation traces as JSON datasets or vector PDF reports directly from the web interface.

# Chapter 9: References & Bibliography

1. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). John Wiley & Sons.

2. Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems* (4th ed.). Pearson Education.
3. Arpaci-Dusseau, R. H., & Arpaci-Dusseau, A. C. (2018). *Operating Systems: Three Easy Pieces* (v1.00). Arpaci-Dusseau Books.
4. Love, R. (2010). *Linux Kernel Development* (3rd ed.). Addison-Wesley Professional.
5. Bovet, D. P., & Cesati, M. (2005). *Understanding the Linux Kernel* (3rd ed.). O'Reilly Media.
6. Stallings, W. (2018). *Operating Systems: Internals and Design Principles* (9th ed.). Pearson.
7. Corbet, J., Rubini, A., & Kroah-Hartman, G. (2005). *Linux Device Drivers* (3rd ed.). O'Reilly Media.